

IBM Bob Multi-Agent AI Software Team

Technical briefing for judges: how it works, why each technology was selected, and how to answer detailed questions.

Problem Statement 5 - Multi-Agent AI Systems

Prepared from the current repository configuration and project evidence

1. Executive summary

This project is a governed multi-agent software-development workflow implemented with IBM Bob custom modes. A small FastAPI to-do REST API is the demonstration application. The core outcome is not CRUD functionality alone: it is a repeatable process that separates research, coding, review, testing, debugging, and documentation into distinct roles with restricted write access.

The central question is: can several specialized AI roles catch failures that one general AI pass might miss? The repository provides concrete evidence: a review found a risky dynamic-SQL pattern, scope drift, and inconsistent validation; a deliberately introduced DELETE response bug was independently detected and diagnosed; and the final API is covered by 23 API tests plus 3 workflow-controller tests.

What is actually being demonstrated?

Layer	What it does	Why it matters
Application	A FastAPI + SQLite to-do API with six endpoints.	Provides a small, runnable proof that the workflow produces working software.
IBM Bob agents	Six custom modes with scoped jobs and file permissions.	Prevents one role from silently doing another role's work.
Workflow controller	Checks evidence, approves valid transitions, and writes audit records.	Makes hand-offs explicit and traceable without falsely claiming to control Bob.
Verification	Independent pytest suite and GitHub Actions CI.	Checks behavior separately from the code-writing role.

2. The problem and solution approach

Why this problem matters

AI coding tools can produce software quickly, but a single general-purpose agent can write code, judge its own code, write narrow tests, and claim completion. That creates self-review blind spots. In a real team, different people plan, implement, review, test, and approve changes because independent perspectives detect different classes of error.

Risk with one general AI	How this project addresses it
Security pattern can survive the first implementation pass.	Review Agent checks security and reports findings without editing source.
Implementation can drift from approved requirements.	Research produces a design brief; Review checks code against the locked scope.
Builder-written tests can miss edge cases.	Testing Agent independently writes tests and cannot change production code.
Raw failures can lead to unfocused fixes.	Debug Agent classifies root cause before Coding Agent makes a targeted repair.

Risk with one general AI	How this project addresses it
No clear record of approval or evidence.	Controller requires named human approval and appends JSONL audit records.

The workflow

Stage	Output artifact	Next decision
Research	design_brief.md	Is the design concrete enough to implement?
Coding	app/ source	Does implementation exist and follow the brief?
Review	review_report.md	APPROVED or CHANGES REQUESTED?
Testing	app/test_main.py and test_report.md	PASS goes to Docs; FAIL goes to Debug.
Debug	debug_report.md	Diagnose APP_BUG, TEST_BUG, or SPEC_MISMATCH.
Pitch/Docs	app/README.md and docs/demo-script.md	Runs only after tests pass.

Normal path: Research -> Coding -> Review -> Testing -> Docs. Failure path: Testing -> Debug -> Coding, followed by review and testing again.

3. Technology choices and rationale

Technology	Used for	Why chosen	Why not the alternative here
Python 3.11	Language and runtime	Readable, fast to develop, excellent FastAPI and test ecosystem.	No need for a compiled stack in a small, local hackathon API.
FastAPI	REST API framework	Typed request/response models, automatic validation, Swagger/OpenAPI docs, and TestClient support.	A lower-level framework would require more validation and documentation work.
Pydantic	Data validation	Validates title length and strict Boolean input at API boundary.	Hand-written validation is more repetitive and easier to make inconsistent.
SQLite via sqlite3	Persistent data store	Built into Python, zero server setup, single-file, reproducible for judges.	PostgreSQL/MySQL add infrastructure that is not required for this local demonstration.
Parameterized SQL	Database queries	Separates values from SQL syntax and makes queries explicit.	An ORM was intentionally out of scope and unnecessary for one table.
pytest + httpx	Automated tests	Fast API-level tests with isolated temporary databases.	Manual tests alone would be slow and not repeatable.
GitHub Actions	Continuous integration	Runs the test suite on pushes and pull requests.	It complements Bob; it does not claim to run Bob agents in the cloud.
IBM Bob custom modes	Specialized AI roles	Persistent role instructions plus enforceable file-scoped edit permissions.	Prompt-only roles depend on the model choosing to obey; scoped permissions are structural controls.

Dependencies

```
fastapi
uvicorn[standard]
pytest
httpx
```

sqlite3 is used directly and is part of the Python standard library, so it is not in requirements.txt.

4. Application architecture and code walkthrough

Data model

Field	Storage type	API type	Rules
id	INTEGER PRIMARY KEY AUTOINCREMENT	integer	Created by SQLite.
title	TEXT NOT NULL	string	Required, non-blank, maximum 200 characters.
done	INTEGER NOT NULL DEFAULT 0	boolean	SQLite stores 0/1; Pydantic returns false/true.

Why 0/1 for done?

SQLite has no separate native Boolean storage class. The application stores the completion value as 0 or 1. The response model defines done as bool, so API consumers always receive true or false rather than the database representation.

Startup and database connections

```
DB_PATH = str(Path(__file__).parent / "todos.db")
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    _init_db(DB_PATH)
    yield
```

On startup, `_init_db` creates the SQLite file and todos table if they do not exist. Each route opens a connection through `_get_conn()`, sets `sqlite3.Row` for named-column access, commits changes where needed, and closes the connection in finally.

API endpoints

Method and path	Purpose	Success	Important validation/error behavior
POST /todos	Create a todo	201 + todo	Title required, non-blank, <= 200 characters; invalid body is 422.
GET /todos	List all todos	200 + array	Empty database returns [].
GET /todos/{id}	Fetch one todo	200 + todo	Missing ID is 404; non-integer ID is FastAPI 422.
PUT /todos/{id}	Update title and/or done	200 + todo	Strict Boolean for done; empty body is permitted as a no-op.
DELETE /todos/{id}	Remove a todo	204 no body	Missing ID is 404.
PATCH /todos/{id}/toggle	Flip done	200 + todo	Missing ID is 404.

Input validation

```
class TodoCreate(BaseModel):
    title: str = Field(max_length=200)

    @field_validator("title")
    @classmethod
    def title_not_empty(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("title must not be empty")
        return v

class TodoUpdate(BaseModel):
    title: Optional[str] = Field(default=None, max_length=200)
    done: Optional[StrictBool] = None
```

Field(max_length=200) handles the length boundary. The validator rejects whitespace-only titles. StrictBool prevents values such as the string 'yes' from being interpreted as true.

5. Security and quality decisions

Safe SQL pattern

```
conn.execute(
    "UPDATE todos SET title = ?, done = ? WHERE id = ?",
    (payload.title, int(payload.done), id),
)
```

All user-supplied values are passed through SQLite placeholders (?) rather than being concatenated into SQL text. The Review Agent previously detected an f-string-created dynamic SET clause. It was not exploitable in the shipped version because field names were internally fixed, but it was a latent risk. The current implementation uses three static update statements: title only, done only, or both.

Real review evidence

- Dynamic-SQL anti-pattern identified and replaced with static parameterized SQL.
- Scope drift found: the toggle endpoint was built before the locked scope document was amended. The scope was versioned to v2.
- Validation inconsistency found: whitespace-only titles were rejected on create but initially could pass on update. The validator was aligned.

Known limitations - be honest

- This is a local proof of concept, not a production deployment.
- No authentication, authorization, tenancy, pagination, rate limiting, monitoring, or deployment configuration are in scope.
- SQLite is appropriate for the demo but not the default choice for a high-concurrency multi-user service.
- Toggle is a read-then-write operation; a production concurrent system would need transaction and concurrency controls.
- Audit records are append-only files, not cryptographically signed or immutable records.

6. How IBM Bob is used

IBM Bob is used through .bob/custom_modes.yaml. Each custom mode defines the agent identity, when it is appropriate to use, expected artifacts, permitted tools, and an edit fileRegex. This is the key difference from six informal prompts: Bob can enforce the permitted edit scope before an agent writes a file.

Agent	Role	Permitted write output	Why the boundary matters
Research	Turns requirements into technical design.	design_brief.md only	Cannot bypass planning by writing implementation.
Coding	Implements approved design or focused fixes.	app/ only	Cannot rewrite test, review, or audit evidence.
Review	Checks security, scope, and quality.	review_report.md only	Reports findings without silently changing source.
Testing	Writes and runs independent tests.	app/test_*.py and test_report.md	Cannot make application changes to force a pass.
Debug	Diagnoses failed tests.	debug_report.md only	Separates diagnosis from fixing.
Pitch/Docs	Writes user-facing documentation after PASS.	app/README.md and docs/demo-script.md	Cannot change the implementation after verification.

What Bob does and does not do

Bob does	Bob does not do
Provide the specialized custom modes, role instructions, tool groups, and file-scoped edit permissions.	Automatically switch modes, approve a hand-off, or claim the whole workflow was run autonomously.
Carry out the assigned role using its allowed capabilities.	Replace a human approval decision.
Block an edit that violates the mode's configured file scope.	Make GitHub Actions run agents in the cloud.

Use this exact judge-safe phrase: 'IBM Bob provides the specialized modes and permission boundaries. A human selects the next mode; our controller validates the evidence and records the approval.'

7. Guarded hand-off controller

workflow/handoff_controller.py is a lightweight governance layer around the manual IBM Bob workflow. It does not impersonate Bob. It reads the repository evidence, checks whether a requested transition is allowed, requires a non-empty approver name, appends a JSONL audit record, and prints the next mode plus required inputs and expected output.

Permitted gates

```
research -> coding -> review -> testing -> docs
|
-> debug -> coding
```

Gate	Evidence required	Result
research -> coding	design_brief.md exists	Coding Agent gets scope and brief.

Gate	Evidence required	Result
coding -> review	app/main.py exists	Review Agent checks the implementation.
review -> testing	review report contains APPROVED	Testing Agent can write and run independent tests.
testing -> docs	test report contains PASS and zero failures	Pitch/Docs Agent creates final documentation.
testing -> debug	test report contains FAIL	Debug Agent diagnoses before any repair.
debug -> coding	debug_report.md exists	Coding Agent receives a focused diagnosis.

Commands to know

```
python workflow/handoff_controller.py status
python workflow/handoff_controller.py handoff review testing --approved-by "Presenter Name"
python workflow/handoff_controller.py handoff testing docs --approved-by "Presenter Name"
```

8. Tests, debugging, and CI

Test approach

The API suite uses FastAPI TestClient. A pytest fixture creates a fresh temporary SQLite database using tmp_path and redirects the application DB_PATH with monkeypatch. This means tests are isolated from the real app/todos.db database and from each other.

Evidence	Count / behavior	Why it matters
API tests	23 passing cases	Covers every endpoint, validation, error conditions, boundary cases, and smoke flow.
Controller tests	3 passing cases	Checks review gate, PASS-to-docs audit creation, and status snapshot.
CI	GitHub Actions runs pytest on push and pull request	Automated independent execution outside the local editor.

Deliberate failure demonstration

To prove the recovery branch, the team changed DELETE /todos/{id} to return HTTP 200 instead of the required HTTP 204. The delete SQL still worked, but the response contract was wrong. test_delete_todo failed with 'assert 200 == 204'. Debug Agent identified both wrong locations: the route decorator and the returned Response. It classified the issue as APP_BUG with high confidence and recommended changing both values back to HTTP 204 NO CONTENT.

This is strong evidence because Debug Agent did more than repeat the test error: it connected the spec, the test, and two source-code locations into a precise repair instruction.

9. Judge questions and ready answers

Q: Why not use one AI agent?

A: One agent can write, review, and test its own work. Separating roles introduces independent checks and reduces self-confirmation bias.

Q: Why six agents?

A: They map to real engineering responsibilities: requirements, implementation, security/scope review, verification, diagnosis, and documentation.

Q: Is this fully automated?

A: No. The human selects the next Bob mode and approves hand-offs. The controller automates evidence checks, permitted transitions, and audit logging.

Q: Why use a to-do API?

A: It is deliberately small and familiar, so the judges can inspect the workflow rather than spend the presentation understanding product complexity.

Q: Why FastAPI?

A: It gives validation, typed schemas, automatic Swagger documentation, and a strong testing interface with minimal code.

Q: Why SQLite?

A: It provides a zero-setup, reproducible local database. For production we would use a managed database such as PostgreSQL.

Q: How are permissions enforced?

A: By IBM Bob custom-mode edit fileRegex scopes, which technically restrict the paths each role can write.

Q: What real issue did the agents find?

A: Review found a dynamic-SQL anti-pattern, scope drift, and a validation gap. Debug diagnosed a deliberately introduced incorrect DELETE status code.

Q: Can the Testing Agent fix a failing API?

A: No. Its scope allows test files and the test report, not app/main.py. Only Coding Agent can change source.

Q: Does GitHub Actions run IBM Bob?

A: No. CI runs the test suite. IBM Bob is used in the development workflow; CI independently validates the repository code.

Q: What would production hardening look like?

A: Authentication, authorization, PostgreSQL, migrations, monitoring, rate limiting, secrets management, protected branches, signed approvals, and immutable audit storage.

Q: What is the key value proposition?

A: AI-assisted development becomes more trustworthy when specialized roles have least-privilege access, independent evidence, and human-approved traceable hand-offs.

10. Final presentation checklist

- Start the API before judges arrive: `python -m uvicorn app.main:app --reload`.
- Open Swagger at `http://127.0.0.1:8000/docs` and create/toggle one todo.
- Open `.bob/custom_modes.yaml` and show Coding and Testing fileRegex restrictions.

- Show workflow status and audit evidence, then review_report.md.
- Run or show the complete test result: 23 API tests + 3 controller tests = 26 passed.
- Show debug_report.md rather than breaking the code live in a short presentation.
- Be precise: the controller does not switch Bob modes; it validates evidence and logs a human-approved hand-off.

Final line to remember: 'We did not build only a to-do API. We built a controlled IBM Bob development team where every role has one job, one permission boundary, and one piece of evidence for the next role.'